

WHAT TO EXPLAIN?

PERSON 1 - Introduction + Overall Flow

What to Speak

Our project is **LAI - Legal Document Intelligence System**. The objective of our project is to automate contract review using Generative AI.

Organizations spend a significant amount of time manually reviewing contracts. Legal teams have to identify risky clauses, compare them against market standards, prepare executive summaries, and answer queries about the contract. This process is slow, expensive, and requires legal expertise.

To solve this, we built an AI-powered system that can analyze contracts end-to-end.

The user uploads a PDF or DOCX contract through the frontend. The request reaches our FastAPI backend, where a contract record is first created in PostgreSQL with the status set to 'uploaded'. After that, our background processing pipeline starts.

The pipeline consists of five major stages:

1. Document Parsing
2. Clause Extraction
3. Risk Scoring
4. Executive Summary Generation
5. Database Storage

Once the analysis is complete, the frontend provides several features like viewing the executive summary, checking individual clause risks, comparing contracts, and chatting with the uploaded contract.

So overall, our system converts an unstructured legal document into structured legal insights that are easy to understand and interact with."

Handover to next person to explain how we convert the uploaded document into structured legal data.

PERSON 2 - Document Parsing + Clause Extraction

What to Speak

"Once the contract is uploaded, the first stage of our pipeline is document parsing.

For parsing, we use **IBM Docling**.

Instead of extracting plain text like traditional PDF parsers, Docling preserves the structure of the document. It maintains headings, section numbers, lists, tables, and formatting, which is extremely important because legal clauses often reference other sections within the contract.

The output generated by Docling consists of:

- Raw text
- Markdown
- Structured JSON representation of the document

This structured output becomes the input to the next stage.

The next stage is Clause Extraction.

Here we use **Gemini 2.5 Flash** with structured JSON output.

We provide the complete parsed markdown document to Gemini, and Gemini identifies important legal clauses such as:

- Indemnity
- Limitation of Liability
- Confidentiality
- Termination

- Intellectual Property
- Payment Terms
- Governing Law

Instead of returning free-form text, Gemini returns structured JSON objects.

We chose structured output because it gives predictable responses that can directly be stored in our database and easily rendered in the frontend.

We also chose an LLM instead of regex because legal clauses can be written in many different formats. Regex can only detect fixed patterns, whereas Gemini understands the semantics of legal language."

Possible Counter Question

Why Docling instead of PyPDF?

Answer:

"Docling preserves document hierarchy, tables, lists and section numbers. This provides much richer context for downstream LLM processing."

Handover to the next person to explain our hybrid risk scoring approach.

PERSON 3 - Hybrid Risk Scoring + Executive Summary

What to Speak

"Once the clauses are extracted, each clause goes through our Hybrid Risk Scoring pipeline.

We combine two different approaches.

The first component is a deterministic rule engine.

Here we use predefined regex rules to identify obvious legal risks.

Some examples include:

- Unlimited liability
- Sole discretion
- Automatic renewal

These patterns always increase the clause risk score.

However, rules alone cannot understand the meaning of complex legal language.

So we introduce the second component, which uses Gemini.

For every extracted clause, Gemini compares it against a corresponding market-standard baseline stored in our PostgreSQL database.

Gemini evaluates how much the clause deviates from the expected market standard and returns:

- Risk score
- Risk explanation
- Market deviation
- Plain English summary

Finally, we combine both scores.

Our final score is:

40% Rule-based score

60% Gemini semantic score.

We selected this hybrid approach because rules provide deterministic behaviour while Gemini provides contextual understanding.

After all clauses are scored, Gemini generates a plain-English executive summary of the entire contract.

This allows even non-legal users to quickly understand the major risks without reading the complete document."

Counter Question

Why not only use Gemini?

Answer:

"LLMs can occasionally be inconsistent. Rules guarantee deterministic detection of obvious legal risks, while Gemini handles complex language."

Handover to next person to explain → all this information is stored in our database and exposed through different product features

PERSON 4 - Database + Features

What to Speak

"After analysis, all results are stored inside PostgreSQL.

We maintain three major tables.

The Contract table stores:

- Contract metadata
- Raw text
- Parsed structure
- Overall risk score
- Executive summary
- Analysis status

The Clause table stores every extracted clause separately.

For each clause we store:

- Original clause text
- Clause type
- Risk score

- Market deviation
- Plain-English explanation

Storing clauses separately avoids repeating extraction whenever users revisit the contract.

The Baseline table stores market-standard clause templates.

These baselines are later used during Gemini's comparison process.

Using this stored information, we provide several frontend features.

The Executive Summary page displays the generated summary.

The Compare Contracts feature compares clauses of the same type across multiple contracts.

Finally, we provide Contract Chat.

For chat, we currently use Long Context Prompting.

Whenever a user asks a question, we send:

- The complete contract
- Clause metadata
- Last five chat messages

to Gemini.

We intentionally did not use RAG for contract chat because legal clauses often depend on definitions present elsewhere in the document. Sending the entire contract prevents losing important context."

Counter Question

Why store clauses separately?

Answer:

"It avoids repeating extraction, speeds up comparison, and enables clause-level analytics."

Handover to the next person to explain the design decisions we made, their trade-offs, and how we plan to improve the

architecture.

PERSON 5 - Design Decisions + Tradeoffs + Future Improvements (3 min)

What to Speak

"While designing the system, we made several architectural decisions based on our current use case.

For document parsing, we selected IBM Docling because it preserves document structure much better than traditional parsers.

For clause extraction, we enforced structured JSON outputs so downstream processing becomes reliable.

For risk scoring, we adopted a hybrid approach combining rules and Gemini because neither approach alone is sufficient.

For market comparison, we currently use baseline clauses stored inside PostgreSQL.

For contract chat, we selected Long Context Prompting instead of RAG.

The reason is that our current application analyzes a single uploaded contract. Since Gemini supports very large context windows, providing the entire contract avoids chunking errors and preserves relationships between clauses.

However, we recognize certain limitations.

Currently:

- We do not use RAG.
- We do not use a Vector Database.
- We do not have persistent chat memory.
- Long-context prompting increases token cost and latency.

In future, we plan to improve the system by:

- Introducing RAG for market-standard comparison using benchmark datasets like CUAD.
- Using Vector Databases for semantic retrieval.
- Supporting cross-contract semantic search.
- Adding LangGraph agents for autonomous legal workflows.
- Introducing persistent memory for chat sessions.

These improvements would make the system more scalable while maintaining the strengths of our existing architecture."

Last 5 Questions Everyone Should Know

Even if the viva is divided, everyone should know these:

1. **Why Docling?**

- Better parsing of headings, tables, lists, and section hierarchy.

2. **Why Hybrid Scoring?**

- Rules provide deterministic detection; Gemini understands legal semantics.

3. **Why Long Context instead of RAG?**

- Single-contract use case. Full context preserves cross-clause relationships. RAG is more beneficial for large document collections and precedent retrieval.

4. **Why PostgreSQL?**

- Strong relational mapping between contracts, clauses, and baselines. Structured storage supports efficient querying and comparisons.

5. **Biggest future improvement?**

- Replace editorial baseline comparison with RAG over a benchmark corpus (such as CUAD) for market-standard comparison, followed by vector search and agentic workflows.